

Letting the Tools Do the Math: A 10.9M-Parameter Language Model on a Graphing Calculator

Anonymous authors

Paper under double-blind review

Abstract

We decided to test the limits by building a physics assistant that runs natively on a TI-Nspire CX II CAS graphing calculator: one ARM926EJ-S core at 396 MHz, with no floating-point unit, a maximum single memory allocation of about 21.5 MiB and no network. We use it as a case study in how the conclusions of a tiny-model evaluation depend on what the evaluation runs. The 10.9M-parameter model never does the arithmetic itself: the runtime picks a relation from a store, puts it in the prompt with the question’s values, and runs the tool call the model writes. Given the correct relation, the model is correct on 96.7% of problems that provide every value; its errors stem from incorrectly copied digits, and an irrelevant value placed before the relevant ones lowers its score from 116 to 61 of 120, a pattern consistent with the training generator’s habit of adding irrelevant values last. End to end, on generated problems, choosing the relation was initially the main limit: the system answered 21 of the same 120 problems correctly and declined most of the rest. Two selection rules that use the values the question assigns raise this to 93 (and to 248 of 300 fresh problems from the same generators), while incorrect answers rise from 2 to 4. A rule-based path through the same runtime with no model is correct more often and incorrect less often on every set that asks for a number; the model declines more reliably. Differences between our harness and the calculator moved strict accuracy by 9 to 21 points and exposed an engine defect that a host-calculator comparison could not detect. A per-token cost model fitted on battery predicts a decode run within 0.4%.

1 Introduction

This paper serves as a case study of how the conclusions of a tiny-model evaluation depend on what the evaluation actually runs. The setting is a graphing calculator, a device many students carry to class. The TI-Nspire CX II CAS has one ARM926EJ-S core at 396 MHz, no floating-point unit and no SIMD, no network, and a maximum single memory allocation of about 21.5 MiB. We built a physics assistant that runs natively on the calculator, on battery, and measured what it costs, where it fails, and what its model contributes.

These limits are what forced the design one way: take out of the weights everything a tiny model is likely to make errors on. Arithmetic goes to an evaluator that runs directly on the calculator. The physics goes to a store of 177 relations, each with its units and conditions. The runtime picks one relation and puts it in the prompt, with the question’s values and the inputs the question gives no value for. The 10.9M-parameter model then only reads the prompt, copies values into a call and writes a sentence or two around the result; otherwise it declines, or explains a relation.

Each time we brought our evaluation closer to the system that actually runs, its result changed. Our contributions are measurements of those steps:

- how the prompt text, the decoding method and the calculator’s int8 engine each moved measured strict accuracy, by 9 to 21 points, and an engine defect that a host-calculator comparison could not detect, found only by scoring on the calculator’s decoder and fixed at a 2.3% speed cost (Section 5);

Table 1: Language models that run on graphing calculators, as their authors describe them. Timings cannot be compared across rows: the workloads, vocabularies and timing methods differ, and only this work states the power state.

	device	how it runs	weights	model	timing reported
CalcGPT 3	TI-Nspire	built-in Python	not stated	8-bit GRU, 640-word vocabulary, dialogue	none
Transformer On Calc	TI-Nspire CX II CAS	native, Ndless	not stated	260K llama2.c, TinyStories	~ 9 (fp32) / ~ 16 (int8) tok/s, 64 tokens
CasioLLM	Casio fx-CG50	native add-in	streamed from flash	third-party, up to 25M, 4-bit	10-token reply in 27.1 s
Our work	TI-Nspire CX II CAS	native, Ndless-derived	in RAM	10.9M, int8, fully trained; tools	1.88 tok/s decode at positions 53–87; 26–75 s per turn from the start of generation; battery

- with the correct relation supplied, what errors the model makes: copying mistakes, and a sensitivity to where an irrelevant value is placed; and a baseline with no model that, with the relation supplied, matches or beats the model on every number and refusal (Section 6);
- in the full system, with the application choosing the relation: where accuracy was lost, counted as correct, incorrect and declined, with a repeat on fresh problems, a negative set whose values bind a relation that does not apply, and the no-model baseline behind the same selector (Section 7);
- a model of the cost of one token by depth, width and position, fitted on battery, and the limits that memory and speed set on model size; each added position costs 40% more with the KV cache (the stored keys and values of earlier positions) filled than empty (Sections 8 and 9).

The assistant itself, in which the model never computes, is the setting (Section 3); Appendix A compares two model widths, with two seeds each.

2 Related Work

Language models on graphing calculators. We are aware of three earlier projects, all released in July and August of 2026 (Table 1): CalcGPT 3 (FredBennett, 2026); Transformer On Calc, the closest in method, which runs a 260K-parameter llama2.c model natively on the same calculator model (stepney141, 2026); and CasioLLM, which streams a quantized model of up to 25M parameters from flash on a Casio fx-CG50 without a third-party loader (samilososami, 2026). We describe them rather than benchmark them, because none measures the same thing: none uses tools or retrieval, and none reports a cost model or a measured memory limit. A TI-84 modification that sends questions over Wi-Fi to a cloud model is different in kind, since no model runs locally on the calculator (ChromaLock, 2024).

Tool use and program-aided reasoning. Letting a model call a tool instead of computing is well established. Toolformer learns when to call an API (Schick et al., 2023); PAL and Program of Thoughts write programs for an interpreter to run (Gao et al., 2023; Chen et al., 2023); Calc-X, ArithmeticGPT, MathSensei and OccamLLM focus on arithmetic (Kadlčík et al., 2023; Liu et al., 2025b; Das et al., 2024; Dugan et al., 2024); and T1 shows small models gaining from handing verification to tools (Kang et al., 2026). Retrieval-augmented generation supplies knowledge the weights lack (Lewis et al., 2020), and picking the correct tool from a large collection can limit later success in ways that a benchmark which supplies the tool hides (Shi et al., 2025). Our selector picks one physics relation with fixed rules rather than retrieving from a large tool set, but comparing a supplied relation with a selected one, as Section 7 does, is the same measurement. Tool use is therefore not our contribution, and neither is the observation that tool-augmented models fail in recognizable ways (Treviño et al., 2025) or that irrelevant information in a problem misleads language models (Shi et al., 2023). What differs is the setting, a 10.9M-parameter model on hardware with

no FPU and no network, and what can be measured there: with the arithmetic handed off, which errors remain, and how they relate to the training data.

Tiny models and scale. TinyStories showed that models of a few million parameters can write fluent text in a narrow domain (Eldan & Li, 2023), and MobileLLM studies design choices below a billion parameters (Liu et al., 2024). Scaling laws relate loss to parameters and data (Kaplan et al., 2020; Hoffmann et al., 2022). Our comparison of two widths asks a narrower question, with data and training steps held fixed: which behaviors a 3.5-fold reduction in parameters removes.

Inference under tight budgets. Work on microcontrollers fits networks into kilobytes to megabytes (Lin et al., 2020; Lai et al., 2018), including tiny language models (Scherer et al., 2024; Yang et al., 2024), and microcontrollers have served as tool endpoints for a model hosted elsewhere (Yang, 2026). We use integer quantization with a scale per group (Jacob et al., 2018; Dettmers et al., 2022). Streaming weights from flash (Alizadeh et al., 2024), as CasioLLM does, avoids a RAM limit at a cost in speed; we keep the weights in memory and discuss the trade-off in Section 9. Our cost model has the same aim as the roofline model (Williams et al., 2009), but it is fitted to measurements on the device rather than derived from peak rates.

Evaluation fidelity. That a model’s evaluation path and its serving path must compute the same values is standard production practice: the ML Test Score asks for a test that training and serving features are identical (Breck et al., 2017), deployed models are known to differ from their reference implementations (Qiu et al., 2022), and inference engines have a documented set of bugs (Liu et al., 2025a). We do not claim that this mismatch is new. We measure how far each difference moved the evaluation of one tiny model, and report a defect that a host-calculator comparison cannot see, because both sides run the same code (Section 5.2).

3 System

3.1 The model never computes

A question passes through four steps (Figure 1).

Selection. The runtime ranks the store’s 177 relations, together with 1,442 glossary terms from the same textbooks (Section 4), against the question, and acts only when it is confident. It is confident when the question writes out a stored relation, or under three rules. The *coverage rule*: the name of the top-ranked relation covers at least three quarters of the question’s content words. The *values rule*: the values the question assigns bind the top-ranked relation, supplying exactly its inputs, and bind no other. The *asked-symbol rule*: among the relations those values bind, exactly one computes the symbol the question asks for. Otherwise it picks nothing, the prompt says that no relation matched, and the model declines. Section 7 measures this step and explains where the last two rules came from.

Assembly. Native code reads the values the question gives ($m = 2$), moves them to the end of the question and builds the prompt: the question, then a record holding the relation, the unit of each variable, the inputs the question gives no value for (*missing*), the conditions under which the relation holds, and any constants.

Generation. The model continues the prompt. When it closes a `<tool>` span, the runtime runs the call and inserts the result as a `<res>` span; the model then writes its answer and ends with `<end>`. The calculator decodes greedily. If the answer states a number more than 2% from the result, the runtime appends it.

Execution. The evaluator supports seven tools: `eval`, `evalat`, `solve`, `diff`, `integ`, `conv` and `stat`. Units are handled only inside `conv`; everywhere else every name is a symbol, so `g` is gravity, not grams.

During training the loss is masked on every `<res>` span, so the model learns to write calls and to restate what it is given, not to predict what the evaluator will return.

3.2 Model and tokenizer

The model is a Llama-2-style decoder (Touvron et al., 2023) with RMSNorm (Zhang & Sennrich, 2019), rotary position embeddings (Su et al., 2024) and a SwiGLU feed-forward block (Shazeer, 2020): width 352,

```

<q>Take What was the motionally induced emf? Given l = 0.057, v = 1.35, B = 0.196../q>
<r>epsilon=B*l*v | epsilon:V B:T l:m v:m/s | missing:none | standard conditions | fit:high
model: <tool> eval<arg> (0.196)*(0.057)*(1.35)</tool>
calculator: <res> 0.0150822</res>
model: <a> 0.01508 V. From epsilon=B*l*v.<end>

```

Figure 1: A generation-and-tool turn logged on the calculator, on battery, with the relation supplied by the test harness rather than chosen by the application’s selector. From the question “Take $l = 0.057$, $v = 1.35$, $B = 0.196$. What was the motionally induced emf?” the runtime built the prompt (moving the values to the end, which leaves the stray “Take” and the double period), ran the model’s call and inserted the result. From the start of generation, including prefill (processing the prompt), the first output appeared after 29.9s and the turn finished after 52.0s; these times exclude prompt assembly and tokenization. The host reproduces this turn token for token (Section 5.3).

six layers, eight heads, feed-forward width 1,024, a context of 512 tokens and a vocabulary of 4,096, with the embedding shared by the output layer (the classifier). That is 10,903,552 parameters in the weight matrices. Weights are stored in 8 bits with one fp32 scale per group of 88; in the file the feed-forward width is padded with zeros to 1,056 so that the group size divides every row (Section 5.2), for 11,629,696 bytes in total.

The tokenizer is trained on our corpus and reserves 11 special tokens for the document structure. The 4,096-entry vocabulary is a speed decision: the classifier reads $\text{width} \times \text{vocab}$ weights for every token, and even at 4,096 it takes 10.6% of a token’s time (Section 8). Scaling that measured stage linearly, we estimate that a 32,000-entry vocabulary would add about 0.31s to every token, 73% of the current total.

Some dimensions are set by the engine rather than being chosen. The group size is fixed when the engine is compiled, because a group size read at run time costs two software divisions per group in the inner loop. With the group size fixed at 88, the width must be a multiple of 88, and rotary embeddings need an even head size. With eight heads that allows widths of 176, 352 and 528, and 528 is wider than the width-440 model that fails to load (Section 9).

3.3 Engine and loader

The inference engine is derived from llama2.c (Karpathy, 2023). Weights are int8 with a scale per group; before each matrix product the activations are converted to int8 per group, and the product is summed in 32-bit integers. Normalization, rotary embeddings, attention and softmax run in fp32 using the compiler’s software floating-point routines, and the KV cache is fp32. The generation loop, tool execution and the 2% answer check are one C module, shared by the application and by the evaluation harness of Section 5. The calculator’s operating system does not run native programs, so the application is started by a loader derived from Ndless (Ndless contributors, 2025), which a setup document installs again after every reset; the hardware is not modified (Appendix B).

4 Training Data

The model is fully trained from scratch on a mixture that is, by tokens, 85% synthetic documents and 15% raw textbook text. The textbook text is every module of 24 OpenStax textbooks (OpenStax, 2026), markup removed; 22 of the books are licensed CC BY-NC-SA 4.0 and 2 CC BY 4.0. No other dataset is used.

The synthetic documents are generated by a program from the record store, whose 177 relations were extracted from the same books and cleaned in passes that removed malformed and non-physics records. Each document is one exchange in the format of Figure 1. The generator draws each variable’s value from a range chosen for that quantity, runs the call with the calculator’s evaluator, and writes the answer from a small set of templates, so the result in every document is the evaluator’s own output. Document types cover answering, declining because a value is missing, declining because no relation matched or because the record does not apply, explaining a relation, defining a term from the books’ glossaries, and symbolic calls. The current version of the corpus has 316,219 documents; the shipped model was trained on an earlier version

from the same generator, and the models of Appendix A on the current one. A check of 400 documents sampled from an earlier 29,982-document version found 12 with a defect as a physics problem, such as mutually inconsistent values or an incorrect choice between answering and declining (3.00%, 95% Wilson interval 1.72–5.17%). The check was done by two AI reading passes whose flags were settled by a third AI pass, not by a person; its false-positive and false-negative rates were not measured, so the interval reflects sampling uncertainty only, and the check predates later fixes, so it does not describe the current version.

One part was produced differently. The explanation documents, one per relation (177 in training), were drafted by an AI model, based on the relation’s OpenStax section for 127 of the 177 relations and with no source section for the other 50, and then corrected by automated checking passes that were themselves AI-assisted. They were not written, edited or reviewed by the authors, and no person has measured their error rate after correction; before correction the drafts contained a physics error 13.4% of the time.

Training runs 8,000 steps at batch 24 and sequence length 512 (98.3M tokens), with AdamW (learning rate 3×10^{-4} , 3% warm-up, cosine decay to 10%, weight decay 0.1) on one laptop GPU. The final training loss is 0.796.

5 Evaluating on the Calculator’s Own Code

Device and power. Every timing in this paper is taken on one TI-Nspire CX II CAS running OS 6.4.0.74, on battery with the USB cable unplugged: with the cable attached the core runs at 288 MHz instead of 396 MHz. Each measuring program reads the clock setting from the PLL and marks a run with the cable attached as invalid. Time is read from a 32.768 kHz crystal timer, which does not depend on the CPU clock.

Host scoring. Quality is scored on a workstation, because one answer takes the calculator up to 75 seconds. Nothing in the scoring path is rewritten for the host. The prompt is built by the calculator’s own C code for reading the question and assembling the record, and in Section 7 by its selection code too. Generation runs the calculator’s int8 engine and generation loop compiled for the host (together, the calculator’s decoder; Apple clang 21.0 at -O2, arm64), decoding greedily, running each call with the calculator’s evaluator and applying its 2% answer check; the calculator’s build uses GCC 14.2 with software floating point.

What is scored. An answerable problem is *correct* if the output is well formed and does not decline, its last tool result is within 0.1% of a reference, and its text states that result to within 2%, the check the calculator applies. A strict version also requires the stated number to be a correct rounding of the result at the precision it shows. The reference is the record’s relation with the question’s values and the store’s constants substituted, run by the same evaluator; we recomputed all 480 references in an independent implementation (Python’s `math`) and every one agreed to within 10^{-8} , which checks the arithmetic, not whether a stored relation is physically correct. Two controls run before every scoring pass: the reference’s own document must be scored correct on every item, and the same document with one value scaled by 1.1 should be scored incorrect, and is on 116 to 119 of every 120 items. Declining is scored strictly: an output that calls a tool is not a refusal. Table 4 lists the item sets.

5.1 How far the harness moves the result

Our first harness scored the fp32 training model, sampled at temperature 0.8, on the evaluation set’s own prompt text. Each of these choices differs from the calculator, and each moved the result (Table 2). The set’s prompt text was written before a change that put the unit of the asked quantity into the record, and it did not move the values to the end of the question as the runtime does; replacing it with the calculator’s own assembly added 21 points. Decoding greedily, as the calculator does, added 12. Moving from fp32 to the calculator’s int8 engine then took 9 points away, and that loss came from a defect.

5.2 A defect only the calculator’s decoder could show

The int8 engine’s quantization and matrix-product routines walk each row in steps of the group size and skip any remainder, which is correct only if the group size divides every row length. The rows of the feed-forward down-projection are as long as the feed-forward width, 1,024, which 88 does not divide: every layer ignored

Table 2: Strict accuracy on the values-given set as the harness is brought closer to the calculator. Each row keeps the changes above it and adds one. The first two rows sample at temperature 0.8 and average three seeds; the rest decode greedily.

harness	strict accuracy
fp32 training model, evaluation set’s prompt text, sampled	55.0%
+ the calculator’s prompt assembly	76.1%
+ greedy decoding, as the calculator decodes	88.3%
+ the calculator’s int8 engine, group 88 as first shipped	79.2%
+ the same engine with every row aligned to a group (shipped)	89.2%

Table 3: Eight turns logged on the calculator, on battery, with relations supplied by the test harness: prompts built by its runtime, tool calls run by its evaluator. The host reproduces each one token for token. Times run from the start of generation and include prefill of prompts of 35 to 83 tokens; they exclude prompt assembly and tokenization. The failures are the ones Section 6 measures at scale.

item set	what the model did	first output	whole answer
values given	copied 2,430 as 243.0 into the call: incorrect	38.4 s	71.0 s
values given	correct	29.9 s	52.0 s
values given	correct (16667.89 stated as 16670)	28.9 s	47.4 s
values given	correct	31.0 s	59.8 s
irrelevant value added	used the irrelevant value in the call: incorrect	26.8 s	51.5 s
value withheld	declined, but named a given variable as missing	36.3 s	43.4 s
no values	declined, naming the missing variable	15.5 s	25.8 s
explain	explained the relation	40.1 s	74.8 s

56 of the 1,024 inputs to that projection, and because the following rows then started off a group boundary, parts of groups were scaled by a neighboring group’s factor. The exporter had chosen 88 because it divides the total length of every tensor, which is a different property. A host-calculator comparison could not see the defect, because both sides ran the same code.

We fixed it without changing the group size. The feed-forward width is padded with zeros to $1,056 = 12 \times 88$: zero rows in the two up-projections and zero columns in the down-projection, which leaves the network’s function unchanged (the largest logit difference in the training framework is 3.5×10^{-6} , and every argmax is identical). The padding costs 2.3% per token, 428.9 against 419.1 ms at the shipped shape. A group size of 32, which also divides every row, cost 27.4% (534.0 ms), because the engine converts one fp32 scale per group in software. The engine now refuses any file whose group size does not divide a row. On the values-given set the fix raised accuracy from 91.7% to 96.7%, and strict accuracy from 79.2% to 89.2%, within a point of the fp32 model’s 88.3%. All later quality results and all timings use the corrected engine; comparisons with the earlier engine are labeled as such.

5.3 Eight generation-and-tool turns agree between host and calculator

A host evaluation tells us about the calculator only if the two produce the same tokens. We logged eight turns on the calculator, on battery (Table 3), with each relation supplied by the test harness: the calculator’s runtime built each prompt, ran the model’s tool calls and inserted the results. The host builds the same eight prompts and reproduces all eight turns, 325 tokens, exactly. This checks prompt assembly, generation and tool execution; it does not run the selector on the calculator, which Section 7 measures on the host.

The host and the calculator do not compute exactly the same numbers. On a probe that walks eight positions, the logits printed by the host and the calculator differ by up to 0.061; but two host builds that differ only in whether the compiler fuses multiply-adds differ from each other by up to 0.073, so bit-for-bit identity is not a meaningful target. The probe’s one disagreement on the top token is a near tie, the top two host logits 0.0027 apart, which also separates host builds: the default build disagrees with the calculator there, and builds with fusion turned off or forced agree with it.

Table 4: The shipped model on the calculator’s decoder (int8, run on the host), greedy, with each item’s own relation supplied. n items (distinct relations). “Correct” means the evaluator’s result is within 0.1% of the reference and the text states it within 2%, the check the calculator applies. “No model” is the rule-based baseline described in the text.

item set: pass requires	n	model	no model
<i>answerable: be correct</i>			
values given	120 (88)	96.7%	100.0%
named by symbol	120 (82)	97.5%	100.0%
named in words	120 (75)	95.8%	100.0%
irrelevant value added	120 (84)	65.0%	99.2%
<i>not answerable: decline, no tool call</i>			
value withheld	120 (87)	100.0%	100.0%
no values	102 (74)	100.0%	100.0%
relation does not apply	120 (79)	0.0%	0.0%
<i>explanation: well formed, states the relation</i>			
explain	87 (71)	93.1%	78.2%

6 With the Correct Relation Supplied

Table 4 scores the shipped model on eight item sets, each built to test one behavior, with each item’s own relation given to it. This is the model’s contribution on its own; Section 7 adds the selection step before it.

When the prompt holds every value, the answers are mostly correct. Given a question that supplies every value the relation needs, the model is correct on 116 of 120 items (96.7%). Under the strict criterion it is 89.2%; the nine items between the two criteria all state a number within 2% of the result, most of them cut off rather than rounded (1299.954 written as “1299”).

The remaining errors are copying errors. All four incorrect answers are calls built with a value copied incorrectly from the question: 2,430 written as 243.0, 272 as 263.0, 9.83 as 8.875 and 390.8 as 390.0. None is a refusal and none is malformed, and the evaluator never made a mistake. What failed was moving digits from the prompt into the call.

Where an irrelevant value is placed matters more than whether it is there. Into each of the 120 values-given questions we inserted one of the generator’s own irrelevant values, after the last relevant value or before the first, and changed nothing else. Placed after, it costs 4 of the 116 correct answers and gains one (113 of 120 correct). Placed before, it costs 56 and gains one (61 of 120 correct), and in 43 of the 59 failures the irrelevant value is in the call. Reversing the order of the relevant values, on the 91 items with two or more, leaves overall accuracy at 87 of 91, with two answers turning incorrect and two turning correct; this argues against a simple fixed-position strategy, but does not show that the order of the values never matters. The sensitivity is consistent with how the training data was written. The generator adds irrelevant values after the relevant ones, and has done so since before the corpus version the shipped model was trained on; in the version the models of Appendix A were trained on, all 75,618 training questions that carry an irrelevant value list it after every relevant value. We did not retrain with irrelevant values in random positions, so this explanation fits the evidence but is not shown to be the cause. The separate irrelevant-value-added set of Table 4 confirms the pattern on items the paired experiment did not touch. Its irrelevant value is shuffled among the relevant ones; split by where it landed, 41 of 41 items are correct with it last, 24 of 47 with it first and 13 of 32 with it in the middle.

Declining works when the prompt says why. When the question leaves out a value the relation needs, the model declines without calling a tool on all 120 items, and on all 102 when it gives no values at all. In both cases the runtime has already written the missing variable into the prompt, so declining only requires reading one field. The refusal names only the variable the prompt lists as missing in 103 of the 120 value-withheld items (85.8%); most of the others name a variable the question did give (Table 3).

Judging fit does not work. When the relation does not answer the question but every one of its variables has a value, nothing in the prompt says so, and the model calls the tool anyway: it declines on none of 120 items. Of the four models of Appendix A, three also decline on none and one declines on 19 of 120; none reliably rejects a relation that does not apply. Choosing the correct relation falls to selection (Section 7).

Explanations pass a format check. Asked to explain a relation, the model writes a well-formed response that states the record’s relation and gives no number that is not in its prompt on 93.1% of items. The relation is already in the prompt, so this measures format, not knowledge or physical truth: a fixed sentence that restates the prompt’s relation would pass it.

On numbers and refusals, a rule-based path with no model does as well or better. The runtime already reads the values, holds the relation and lists what is missing, so we also ran the simplest system with no model: decline when a value is missing, otherwise evaluate the relation and state the result, and for a question worded as a request for an explanation, show the stored one. On the same prompts and graders (Table 4) it is correct on every answerable item but one, where a parser defect binds a stated Q to the constant q and gives the model the same faulty prompt. It declines on every item that lacks a value, declines on none of the relation-does-not-apply set, and passes the explanation format check on 78.2%. With the relation supplied, then, the runtime’s own path of reading, binding and evaluating matches or beats the model on every item set that asks for a number or a refusal. The model writes the tool arguments and the text, and its measured failures are copying errors in the arguments and imprecise restatements of results. Section 7 repeats this comparison behind the selector.

A smaller model. Appendix A trains 3.1M- and 10.9M-parameter models the same way on the current corpus (new models, not the shipped one), with two seeds each. The smaller model declines just as often when the prompt names a missing value (99.6% at both widths), is correct 19.6 points less often when every value is given, mostly because of incorrect values in the call, and passes the explanation format check 34.5 points less often. Two seeds per width are not a variance estimate, and equal steps do not hold convergence constant.

7 End to End: Choosing the Relation

Section 6 gives the model each item’s own relation. The deployed application chooses the relation itself (Section 3), so the answer a student receives depends first on that choice. We measure both steps together by running the application’s own selection code on the host (the function the enter key calls, with its display stubbed out), then the calculator’s decoder as before, on the same items, and report each outcome: correct, incorrect, or declined. An answerable item is correct under the criterion of Section 5; an item that should be declined counts as correct when the output declines; an explanation is correct only if the correct relation is chosen and the format check passes. Four sets should be declined: the three of Section 6, and 2,000 textbook questions that an automatic criterion certifies no stored relation answers. One of the three, the relation-does-not-apply set, is also a negative set here: its questions assign values that bind one relation exactly while asking for a different quantity, which is where a rule that trusts values could choose confidently and incorrectly. The textbook set also supplies the questions of the training documents that teach declining, so it is an in-distribution check of refusal, not an unseen one.

Selection caused most of the original system’s failures. With the selector as first deployed, the application was correct on 21 of the 120 values-given problems and declined 97, against 116 correct for the model alone; of the 22 questions for which it chose the correct relation, 21 were answered correctly (Table 5). The ranking itself was mostly correct: on questions naming the quantity in words, its top choice was the correct relation 88.3% of the time, but only 30.0% of those questions received any relation and 28.3% the correct one. The coverage rule asks how much of the question the name of the top-ranked relation accounts for. It had been tuned on name lookups, textbook word problems and out-of-scope questions, never on questions that carry values, and every framing word such as “Take” or “Estimate” counts against it.

Values assigned in the question give a second selection signal. The values rule makes the selector confident when the values the question assigns are exactly the inputs of the top-ranked relation: every value names one of its variables, every variable except the one it computes is given or is a stored constant, and

Table 5: The full system on the calculator’s decoder, with the application choosing the relation. Answerable sets and explanations: correct / incorrect / declined. Sets that should be declined: declined / answered. “Model alone” supplies each item’s relation (Table 4). The selector columns show the selector as first deployed (the coverage rule), then with the values rule added, then with the asked-symbol rule added (Section 3); “no model” runs the rule-based path of Section 6 behind the current selector. Fresh sets: 300 new problems per set from the same generators with unused seeds.

item set	n	model alone	selector			no model
			coverage	+ values	+ asked symbol	
<i>development sets</i>						
values given	120	116	21 / 2 / 97	59 / 4 / 57	93 / 4 / 23	96 / 1 / 23
named by symbol	120	117	0 / 0 / 120	27 / 1 / 92	115 / 3 / 2	118 / 0 / 2
named in words	120	115	32 / 2 / 86	89 / 4 / 27	89 / 4 / 27	93 / 2 / 25
irrelevant value added	120	78	9 / 4 / 107	9 / 4 / 107	9 / 4 / 107	13 / 2 / 105
explain	87	81	37 / 3 / 47	37 / 3 / 47	37 / 3 / 47	26 / 3 / 58
decline: value withheld	120	120	120 / 0	119 / 1	119 / 1	116 / 4
decline: no values	102	102	102 / 0	102 / 0	102 / 0	93 / 9
decline: relation does not apply	120	0	120 / 0	120 / 0	119 / 1	118 / 2
decline: out of scope	2,000		1,982 / 18	1,982 / 18	1,982 / 18	1,981 / 19
<i>fresh sets</i>						
values given	300		48 / 2 / 250	153 / 5 / 142	248 / 5 / 47	253 / 2 / 45
named by symbol	300		2 / 3 / 295	79 / 6 / 215	285 / 10 / 5	295 / 0 / 5
named in words	300		63 / 4 / 233	200 / 9 / 91	200 / 9 / 91	209 / 2 / 89
irrelevant value added	300		23 / 18 / 259	23 / 18 / 259	23 / 18 / 259	40 / 8 / 252

the computed one is not given. The rule stays silent when the same values bind a second relation (two resistances bind both series and parallel resistance) and when the question names a different variable from the store (“ $P = 464$, $A = 3.39 \times 10^{-5}$, compute v_d ” binds $I = P/A$ but asks for a drift velocity). The letters a , A and I count as symbols only at the end of a clause, since elsewhere they are usually English words.

The symbol the question asks for settles ties. The values rule’s caution made it silent on questions such as “Given $m = 2$, $a = 3$, find F ”, whose values bind both $F = ma$ and $F_{\text{net}} = ma$, and “Given $V = 12$, $I = 2$, find R ”, whose values bind both $R = V/I$ and $P = VI$. The asked-symbol rule takes, among the relations the values bind, those whose computed variable the question names, and chooses that relation if there is only one; two named relations remain ambiguous.

More answers, and more incorrect ones. Across the four answerable development sets, correct answers rose from 62 to 306 of 480 and declines fell from 410 to 159, while incorrect answers rose from 8 to 15, because more questions now reach the model and some of them meet its copying errors. Incorrect choices of relation did not rise on any answerable set and fell on two, from 6 to 3 and from 6 to 0, because a named symbol overrides an incorrect choice based on words. Of the current selector’s eight incorrect choices on these sets, seven are declined and one produces an incorrect answer: a question asking for “equivalent resistance” without saying series or parallel. Every other incorrect answer follows a correct choice and comes from a copying error.

Values that bind a relation that does not apply. No version of the selector chose the inapplicable relation on the negative set, and the rules did not change the out-of-scope set, whose questions assign no values and so cannot trigger them. The current selector’s one answer on the negative set (“Estimate omega. Measurements give $v = 24.7$, $r = 1.74$ ”) used $\omega = v/r$, which does answer that question, and so did its one answer on the value-withheld set ($I = P/(4\pi r^2)$ for “ $r = 30$, $P = 5.33$, what is I ?”), whose item withholds A for $I = P/A$. Of the 2,000 out-of-scope questions, 32 drew a confident choice and 18 were not declined: 10 got a glossary definition, 7 an explanation of a relation and 1 a computed answer.

On problems the rules were not designed on. We wrote both rules while reading the failures on these items, so their gains on them are not held-out estimates. On 300 fresh problems per set the pattern repeats (Table 5): correct answers went from 48 to 248 when values are given, from 2 to 285 when the quantity is named only by symbol and from 63 to 200 when it is named in words, with incorrect answers going from 2 to 5, from 3 to 10 and from 4 to 9. An irrelevant value blocks both rules, since every value must belong to

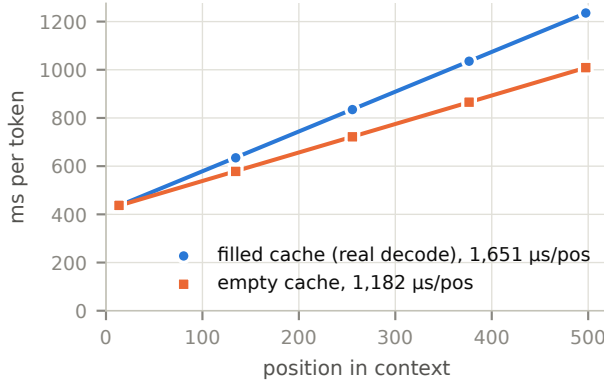


Figure 2: Cost per token against position, shipped shape, battery; each point averages 12 positions. Filling the KV cache raises the slope by 40%.

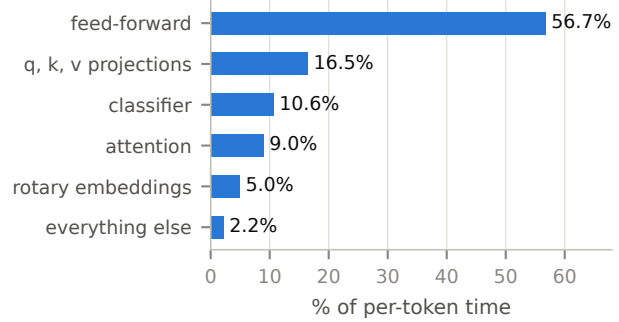


Figure 3: Where one token’s 428.8ms goes, shipped shape, positions 3 to 14, battery. Attention’s share grows with position.

the relation, so that set is unchanged. This controls for tuning to particular items, not for the generator’s templates or its use of the store’s own symbols, which the fresh problems share.

What the model adds in the full system. Behind the current selector, the no-model path is correct more often and incorrect less often on every answerable set, development and fresh; on the fresh named-by-symbol problems it is correct on 295 with none incorrect, against the model’s 285 and 10. The no-model path declines less reliably. On 12 of the 222 value-withheld and no-values items the selector chose a glossary term, such as “null measurements” for a question beginning “Measurements give” or the glossary entry for “compute the equivalent resistance”, and the no-model path shows the definition where the model declines; its keyword rule for explanation requests also passes the format check on 26 of 87 items against the model’s 37. On these benchmarks the model adds no numerical accuracy; it declines more reliably when selection fails, and it writes the text, whose quality and teaching value we did not measure.

8 What a Token Costs

We measure the cost of one decoded token by depth, width and position. Depth, position and the breakdown by stage come from one battery run of one benchmark. It times windows of 12 consecutive positions and reports their mean, which we place at the window’s mean position. Width comes from a sweep in that session.

Depth and width. With the weights fixed and the number of layers varied from one to six, at positions 3 to 14, the cost per token is $46,102 + 63,777L$ microseconds ($R^2 = 1.000000$). The intercept is the fixed cost per token of the embedding lookup, the classifier and the final normalization. Across three shapes of 3.1, 6.4 and 10.9 million parameters, timed over positions 11 to 18, the cost is $32,338 + 37,474N$ microseconds for N million parameters ($R^2 = 0.99983$).

Position. At the shipped shape, with the KV cache (the stored keys and values of earlier positions) filled by decoding every earlier position, the cost is $413,753 + 1,651p$ microseconds at position p ($R^2 = 0.999992$; the intercept is an extrapolation to $p = 0$). That is 1.60 tokens/s at position 128, 1.20 at 256, and 0.81 over the last measured window, at a mean position of 497.5; the fit extrapolates to 2.42 at position 0 and 0.80 at 511, a 67% drop across the context (Figure 2). A speed given without its position can be off threefold.

Where the time goes. Timing each stage of a token at positions 3 to 14 accounts for all but 0.28% of the measured 428.8ms (Figure 3). Matrix products take 83.9% of it: the feed-forward block 56.7%, the query, key and value projections 16.5% and the classifier 10.6%. Attention takes 9.0% and rotary embeddings 5.0%. The parts agree with each other within the same run and at the same positions: the fixed stages are within 0.15% of the depth intercept, the per-layer stages within 0.31% of the depth slope, and all stages together within 0.03% of the position model at their mean position.

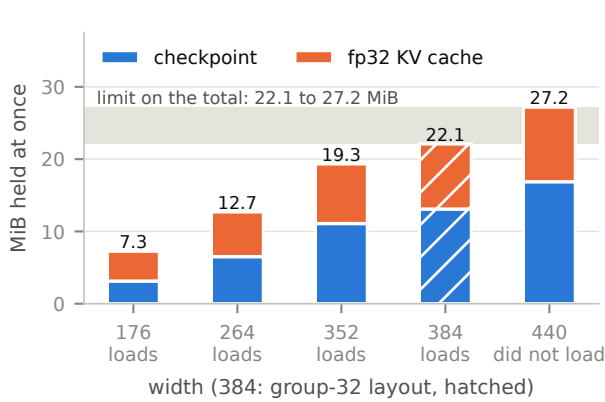


Figure 4: Checkpoint and fp32 KV cache per width (six layers, context 512), and whether the model loaded. The shaded band is where the limit on the two-block total must lie; width 384 is a group-32 layout. The largest single block directly after a reset was 21.53 MiB.

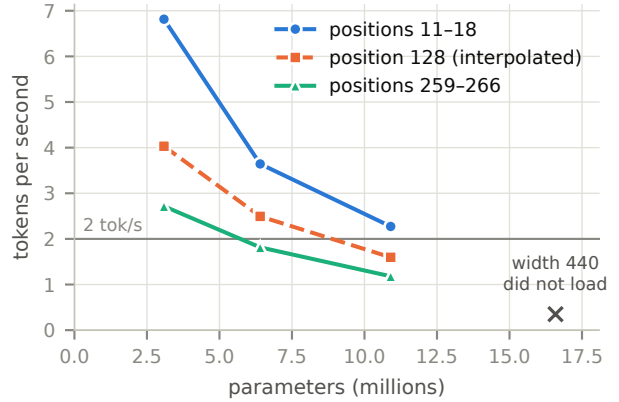


Figure 5: Speed against parameters, battery, filled cache, feed-forward width padded to a multiple of 88. Solid: positions 11–18 and 259–266 (Table 6); dashed: position 128, interpolated. Widths 176 and 264 use random weights.

Prediction. Adding up the position model over the positions of a real decode predicts the decode time of a run it was not fitted to, recorded by a different program: 18.53s predicted against 18.60s measured for 35 tokens at positions 53 to 87 (−0.4%), a decode speed of 1.88 tokens/s. This is one run of one prompt, so it checks the per-token model, not a varied workload, and it excludes prefill (processing the prompt before the first output token), which took 24.3s for 53 tokens. Three decode runs on the engine before the fix of Section 5.2 ranged from 1.920 to 1.923 tokens/s.

An empty cache measures a cheaper computation. Our first benchmark reached position p by jumping there, leaving the cache rows below p unwritten and therefore zero. With the input token held fixed, so that the cache contents are the only difference, filling the cache raised the fitted cost per position by 40%, from 1,182 to 1,651 μ s, with the intercept within 1.6%. The likely reason is that without an FPU every multiply-add and exponential in attention is a software routine, and those routines take shortcuts when an operand is zero; we have not checked this at the instruction level. The empty-cache model under-predicts the decode run above by 5.3%.

9 Memory and Speed Limits

Memory. Directly after a reset, the largest single block an application can allocate is 21.53 MiB. The checkpoint is one block, but it is not the only constraint: the engine also holds an fp32 KV cache of $2 \cdot L \cdot \text{context} \cdot \text{width} \cdot 4$ bytes. At the shipped shape the cache is 8.25 MiB, 43% of the 19.34 MiB that checkpoint and cache take together (Figure 4). The next width in the same layout, 440 with ten heads and 16.6M parameters, needs 27.19 MiB for the two and fails to load on the device, although its 16.87 MiB checkpoint alone would fit in one block. A 12.2M-parameter model of width 384, in a group-32 layout, did load, with 22.10 MiB in its two blocks. For six layers, a 512-token context and an fp32 cache, the limit therefore lies between 12.2M and 16.6M parameters, and because the cache grows linearly with context length, context and parameters trade off directly.

The largest block also depends on what the device did recently: after a program failed inside the loader it was 5.02 MiB until the calculator was reset, while four load-and-free cycles of the shipped model left it unchanged to the byte, so the loss comes from the crash, not from fragmentation.

Speed. Timing each width with a filled cache gives Table 6 and Figure 5. Over positions 11 to 18 the largest tested configuration that reaches 2 tokens/s is the shipped 10.9M; at position 128, by linear interpolation

Table 6: Speed and memory by width on battery, filled KV cache, context 512. Tokens/s averaged over positions 11–18 and 259–266. Widths 176 and 264 use random weights; the feed-forward width of every file is padded to a multiple of 88, as in the shipped one.

	width	params	file + KV	11–18	259–266
	176	3.09M	7.26 MiB	6.82	2.72
	264 (6 heads)	6.40M	12.70 MiB	3.64	1.82
	352	10.90M	19.34 MiB	2.27	1.18
	440 (10 heads)	16.59M	27.19 MiB	does not fit	

between the two measured windows, it is the 6.4M width (about 2.5 tokens/s); over positions 259 to 266 it is 3.1M. At short context, memory and speed set the same limit on width.

When random weights are valid. Timing untrained widths with random weights is valid only if decode time does not depend on the weights’ values. At width 352 a random-weight model and the trained one differ by 1.2% over positions 11 to 18 and by 3.9% over 259 to 266, consistent with software floating point’s data-dependent timing. The threshold results above would survive differences of that size, but the comparison does not bound weight-dependent variation at the other widths.

Flash as the alternative. Streaming weights from flash, as CasioLLM does (samilososami, 2026), avoids the RAM limit, but on this calculator sequential flash reads run at about a forty-eighth of the DRAM read speed, so we keep the model in memory.

10 Limitations

The evaluation is synthetic. Every item, including the fresh ones of Section 7 but not the 2,000 out-of-scope questions, comes from the generators that produced the training documents and uses the store’s own symbols, with values the model has not seen. How often students’ own wording and notation lead to a confident choice is not measured. Almost every call in the evaluation is to `eval`, one of the seven tools, and we do not report perplexity. The model cannot judge whether a relation fits (Section 6), so a question answered with an incorrect relation, such as the ambiguous resistance question, gets a confident incorrect answer.

One device, and few repeats. All timings come from one calculator. On the shipped engine the decode speed was measured once, as were most other timing cells. Appendix A compares only two widths.

Explanations are scored for form, not truth. Whether an explanation’s physics is correct is not measured, and the explanation training text has not been reviewed by a person (Section 4).

11 Conclusion

Given the correct relation, a 10.9M-parameter model that never computes is correct on 96.7% of fully specified physics problems on the calculator’s own decoder, and its incorrect answers are all copying errors. A rule-based path through the same runtime with no model answers all of these problems correctly, and behind the application’s own selector it is still correct more often and incorrect less often, while the model declines more reliably when selection fails. Each step that brought our evaluation closer to the deployed system changed its result: one exposed an engine defect that a host-calculator comparison could not see, and the last showed that most of the original system’s accuracy was lost before the model ran, in a step with no neural network. On hardware this limited, the evaluation worth reporting is the one that runs the device’s own code, end to end, and counts incorrect answers and declines as well as correct ones.

Broader Impact Statement

A physics assistant on a device allowed in some educational examinations could be used to answer test questions. The system chooses the relation itself when it is confident and declines otherwise, but a student can still get a confident incorrect answer when a question is ambiguous or the chosen relation does not apply, which the model cannot detect. The explanation texts it learned from were drafted by an AI model and have not been reviewed by a person. The store, the corpus and the weights will be released, so an institution can inspect what the assistant contains and decide whether to allow it.

Statements and Declarations

- **Funding.** No external funding was received for this work.
- **Competing interests.** The authors declare no competing interests.
- **Data availability.** The record store, the training corpus, the trained weights, and the device logs and results behind every table and figure will be released with the final version of this paper. The corpus, the store and the weights will be released under CC BY-NC-SA 4.0, following the licenses of the OpenStax source text. A video of the calculator in use (Appendix B) is in the supplementary material.
- **Code availability.** The code, including the calculator application, the inference engine, the corpus generator and the evaluation harness, will be released with the final version of this paper, under the MIT License for the code written for this work and the Mozilla Public License 1.1 for the loader derived from Ndless.
- **Use of AI tools.** Anthropic’s Claude models, Claude Opus 5 and Claude Opus 5.5, used through the Claude Code command-line interface, helped automate the software and experimental tooling, ran components of the host-side experiments and analyses, and drafted the text. The authors directed the work, carried out the measurements on the calculator, verified all results and chose what to report.
- **Trademarks.** TI-Nspire is a trademark of Texas Instruments, and Casio and fx-CG50 are trademarks of Casio Computer Co.; this work is not affiliated with or endorsed by either company.

References

- Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL <https://aclanthology.org/2024.acl-long.678>.
- Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the IEEE International Conference on Big Data*, 2017. URL <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=YfZ4ZPt8zd>.
- ChromaLock. TI-32: ChatGPT on a TI-84 via an added ESP32. GitHub repository, 2024. URL <https://github.com/ChromaLock/TI-32>. Created 4 August 2024; accessed 24 September 2026.
- Debrup Das, Debopriyo Banerjee, Somak Aditya, and Ashish Kulkarni. MATHSENSEI: A Tool-Augmented Large Language Model for Mathematical Reasoning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024. URL <https://aclanthology.org/2024.naacl-long.54>.

- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c3ba4962c05c49636d4c6206a97e9c8a-Abstract-Conference.html.
- Owen Dugan, Donato M. Jiménez-Benetó, Charlotte Loh, Zhuo Chen, Rumen Dangovski, and Marin Soljačić. OccamLLM: Fast and Exact Language Model Arithmetic in a Single Step. In *Advances in Neural Information Processing Systems*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/3eceb70f47690051d6769739fbf6294b-Abstract-Conference.html.
- Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English?, 2023. arXiv:2305.07759v2.
- FredBennett. CalcGPT 3: Experimental offline neural language model for the TI-Nspire. Cemetechn file archive, 2026. URL <https://www.cemetechn.net/downloads/files/3024/x4069>. Posted 19 July 2026; accessed 24 September 2026.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. In *International Conference on Machine Learning*, 2023. URL <https://proceedings.mlr.press/v202/gao23f.html>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c1e2faff6f588870935f114ebe04a3e5-Abstract-Conference.html.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html.
- Marek Kadlčík, Michal Štefánik, Ondrej Sotolar, and Vlastimil Martinek. Calc-X and Calcformers: Empowering Arithmetical Chain-of-Thought through Interaction with Symbolic Systems. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12101–12108, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.742.
- Minki Kang, Jongwon Jeong, and Jaewoong Cho. T1: Tool-integrated Verification for Test-time Compute Scaling in Small Language Models. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/776a5f2c7d6dd4b0d83145fc044e2726-Abstract-Conference.html.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models, 2020. arXiv:2001.08361v1.
- Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. GitHub repository, MIT license, 2023. URL <https://github.com/karpathy/llama2.c>. Accessed 24 September 2026.
- Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs, 2018. arXiv:1801.06601v1.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing*

- Systems*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny Deep Learning on IoT Devices. In *Advances in Neural Information Processing Systems*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/86c51678350f656dccc7f490a43946ee5-Abstract.html>.
- Mugeng Liu, Siqi Zhong, Weichen Bi, Yixuan Zhang, Zhiyang Chen, Zhenpeng Chen, Xuanzhe Liu, and Yun Ma. A first look at bugs in LLM inference engines. arXiv:2506.09713v2, 2025a. URL <https://arxiv.org/abs/2506.09713>.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning*, 2024. URL <https://proceedings.mlr.press/v235/liu24ce.html>.
- Zitao Liu, Ying Zheng, Zhibo Yin, Jiahao Chen, Tianqiao Liu, Mi Tian, and Weiqi Luo. ArithmeticGPT: empowering small-size large language models with advanced arithmetic skills. *Machine Learning*, 114(24), 2025b. doi: 10.1007/s10994-024-06681-1.
- Ndless contributors. Ndless: native code for the TI-Nspire. GitHub repository, MPL 1.1, commit 9484d8d, 2025. URL <https://github.com/ndless-nspire/Ndless>. Accessed 24 September 2026.
- OpenStax. OpenStax textbook source repositories. GitHub, twelve `osbooks-*` repositories containing 24 textbooks; CC BY-NC-SA 4.0 (22 books) and CC BY 4.0 (2 books), 2026. URL <https://github.com/openstax>. Accessed 25 September 2026.
- Hang Qiu, Ioanna Vavelidou, Jian Li, Evgenya Pergament, Pete Warden, Sandeep Chinchali, Zain Asgar, and Sachin Katti. ML-EXray: Visibility into ML deployment on the edge. In *Proceedings of Machine Learning and Systems*, volume 4, 2022. URL https://proceedings.mlsys.org/paper_files/paper/2022/hash/c04f25be56ab86371563568dce31808f-Abstract.html.
- samilososami. CasioLLM: Offline language models on Casio fx-CG50 graphing calculators. GitHub repository, GPL-3.0, 2026. URL <https://github.com/samilososami/CasioLLM>. Created 15 August 2026; accessed 24 September 2026.
- Moritz Scherer, Luka Macan, Victor J. B. Jung, Philip Wiese, Luca Bompani, Alessio Burrello, Francesco Conti, and Luca Benini. Deeploy: Enabling Energy-Efficient Deployment of Small Language Models on Heterogeneous Microcontrollers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(11):4009–4020, 2024. doi: 10.1109/TCAD.2024.3443718.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://proceedings.neurips.cc/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html>.
- Noam Shazeer. GLU Variants Improve Transformer, 2020. arXiv:2002.05202v1.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31210–31227, 2023. URL <https://proceedings.mlr.press/v202/shi23a.html>.
- Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 24497–24524, 2025. doi: 10.18653/v1/2025.findings-acl.1258. URL <https://aclanthology.org/2025.findings-acl.1258/>.

- stepney141. The world of calculator programming: Transformer On Calc. Talk (in Japanese), Kernel/VM Tokyo No. 19, 2026. URL <https://image.docswell.com/s/stepney141/5GN6DJ-kernel-vm-tokyo-19>. Given 22 August 2026; accessed 24 September 2026.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open Foundation and Fine-Tuned Chat Models, 2023. arXiv:2307.09288v2.
- Eduardo Treviño, Hugo Contant, James Ngai, Graham Neubig, and Zora Zhiruo Wang. Benchmarking failures in tool-augmented language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 2916–2934, 2025. URL <https://aclanthology.org/2025.naacl-long.149/>.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- Dongxu Yang. Device Context Protocol: A Compact, Safety-First Architecture for LLM-Driven Control of Constrained Devices, 2026. arXiv:2605.26159v1.
- Zebin Yang, Renze Chen, Taiqiang Wu, Ngai Wong, Yun Liang, Runsheng Wang, Ru Huang, and Meng Li. MCUBERT: Memory-Efficient BERT Inference on Commodity Microcontrollers. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–9, 2024. doi: 10.1145/3676536.3676747.
- Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems*, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.

A Two Model Widths

We trained a 3.1M-parameter model of width 176 exactly as we trained one of width 352: the same corpus version and tokenizer, six layers, eight heads, 8,000 steps at batch 24 and the same learning-rate schedule, with two training seeds at each width. Width is the only difference. The two widths were chosen under the group-88 layout, so the comparison has two points and we fit nothing through them. All four models are scored as in Section 6, with the relation supplied, on the calculator’s decoder with a group size of 16, which divides every row at both widths. The shipped layout differs from group 16 by at most 1.7 points on the shipped model.

Declining is unchanged. Declining when the prompt names a missing value is not affected by the 3.5-fold reduction in parameters: 99.6% at both widths, and 100% when no values are given at all (Table 7).

Table 7: The two widths on the calculator’s decoder (int8, run on the host), group 16, greedy, relation supplied. Each cell is the percentage of items passed by one training seed (s1, s2); the change is between the two-seed means, in points. Items per set as in Table 4. The explanation row is the format check.

item set	3.1M		10.9M		change
	s1	s2	s1	s2	
decline: value withheld	99.2	100.0	99.2	100.0	0.0
decline: no values	100.0	100.0	100.0	100.0	0.0
correct: values given	65.0	83.3	95.0	92.5	−19.6
correct: named by symbol	70.8	77.5	99.2	98.3	−24.6
correct: named in words	75.0	80.8	96.7	95.8	−18.3
correct: irrelevant value added	41.7	55.8	63.3	52.5	−9.2
explanation (format)	64.4	52.9	93.1	93.1	−34.5
decline: relation does not apply	0.0	0.0	0.0	15.8	−7.9

Numeric answers get worse, in the call. Accuracy falls by 18 to 25 points on the three answerable sets without an extra value, from 93.8% to 74.2% when every value is given. The loss is in the call. Calls built with an incorrect value are 1.7% and 5.0% of items at width 352 and 32.5% and 13.3% at width 176, while failures of the strict criterion stay between 8% and 18% at both widths. With an extra value present both widths do poorly and the gap narrows to 9 points.

Explanation format drops the most, and judging fit fails at both widths. Passing the explanation format check falls from 93.1% to 58.6%, the largest drop in the table. Declining a relation that does not apply is at or near zero throughout; the one exception, 15.8%, comes from the second seed at width 352, which also scored highest on this set under earlier scoring, and we read it as seed variation rather than a capability.

One description consistent with these numbers is that the accuracy drop grows with how much the model must produce beyond what the prompt already contains: a one-line refusal, a call that copies several values, a paragraph of text. We did not vary that amount with the task held fixed, so it is a description, not a mechanism. The width-176 seeds differ by up to 18 points, and two seeds are not a variance estimate; equal optimizer steps do not hold convergence constant either (final losses 0.963 and 0.952 at width 176, 0.799 and 0.787 at 352). Predictions we registered before training named an item set we later found measures declining, scored by a harness we later replaced (Section 5.1), so this experiment is not a test of them.

B The Calculator in Use

Figure 6 shows three frames from a video of the calculator answering the question of Figure 1, filmed in one take on battery with no cable attached. ChatTLM Setup, opened after the reset button on the back of the calculator was pressed, installed the loader and closed, and the home screen reported it (a). The question was then typed on the keypad, whose letter keys are in alphabetical order, and entered. This time the application chose the relation itself, and it printed the same answer as in Figure 1, complete about 59 s after the question was entered (b, c). That time is read from the video; unlike the 52.0 s of Figure 1, it includes choosing the relation, building the prompt and tokenizing it. The video, unedited apart from re-encoding, is in the supplementary material.

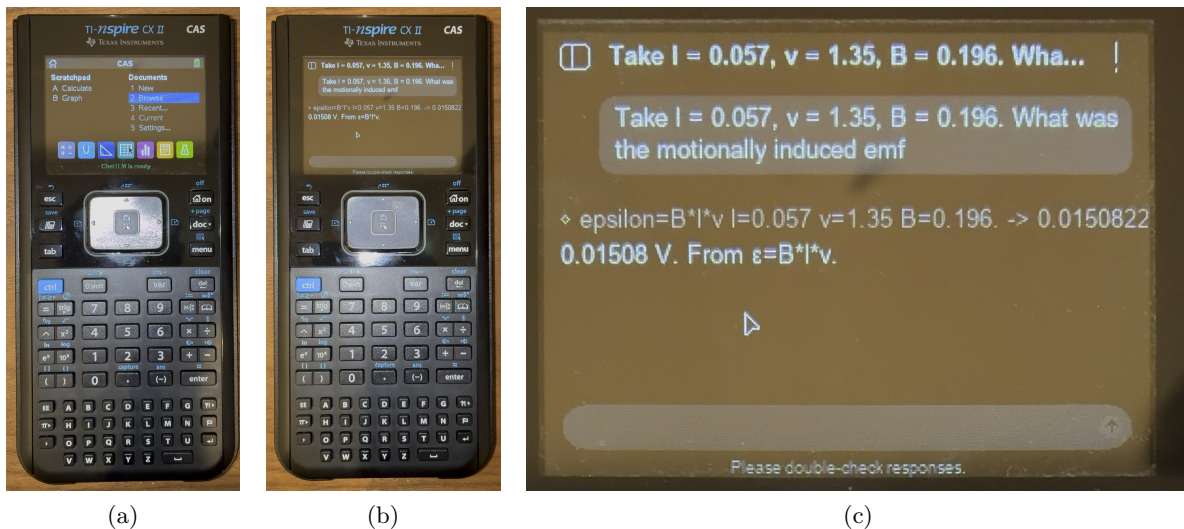


Figure 6: Frames from the video, cropped and scaled but not retouched. (a) The home screen after ChatTLM Setup has run; the green line reads “ChatTLM is ready” and the battery symbol is at the top right. (b) The finished answer. (c) The screen of (b), enlarged: the question, the relation the application chose with the values it read and the result of the call ($\epsilon=B \cdot I \cdot v$ $I=0.057$ $v=1.35$ $B=0.196$. $\rightarrow 0.0150822$, with the stray period noted under Figure 1), and the answer.

Figure 7 shows four screens drawn by the application’s own drawing code, compiled for a computer, at the calculator’s 320×240 pixels. The application chooses the relation and builds the line above each answer exactly as it does on the calculator, and each answer is the output of the calculator’s decoder run on the host, which reproduced all eight turns logged on the calculator token for token (Section 5.3). Screen (b) is the one filmed in Figure 6c.

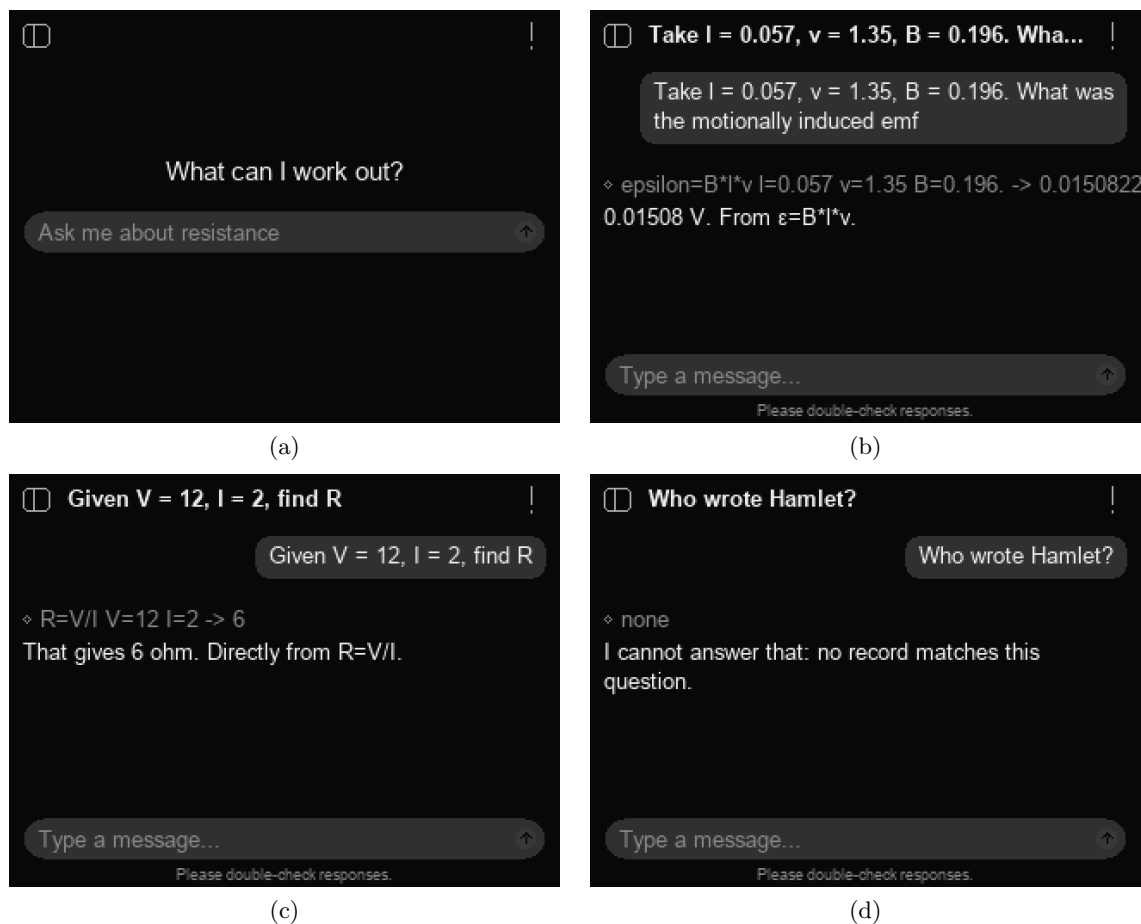


Figure 7: Screens drawn by the application’s own code, in the dark theme the calculator was using when it was filmed (the theme follows the clock). (a) The start screen. (b) The question of Figure 6, as it was typed. (c) A question that names the quantity it asks for by symbol. (d) A question no record answers, which the application declines. Above each answer is the relation the application chose (“none” in (d)), with the values it read and the result of the call.